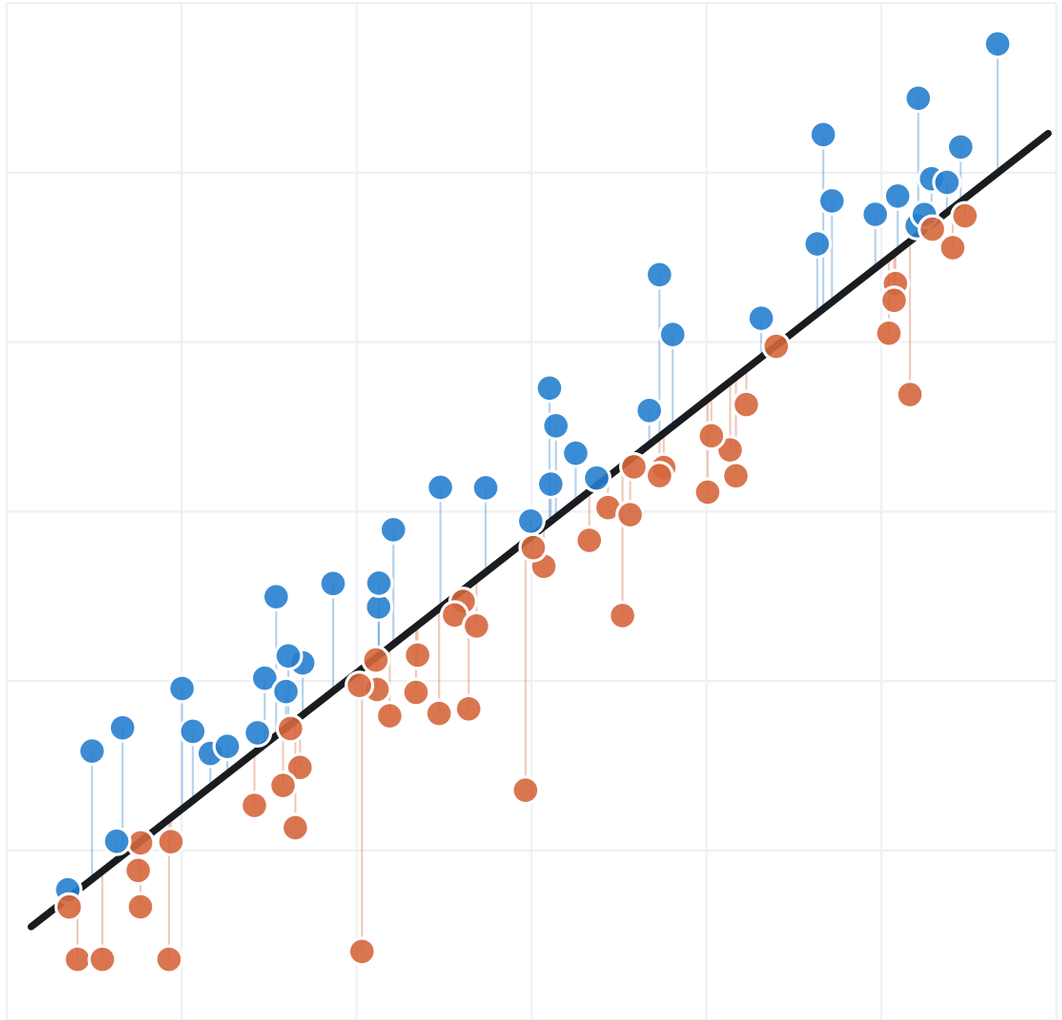
Линейные модели



Линейная и логистическая регрессия: от прямой по точкам до вероятности класса

Т-поколение ИИ · базовая ML-группа

15 ноября 2026

ЧАСТЬ 1

Линейная регрессия

Раньше прогноз собирали из деревьев. Теперь соберём его из признаков, умноженных на веса.

От суммы деревьев к сумме вкладов признаков

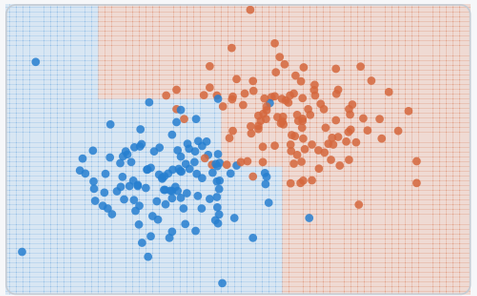
Меняем семейство моделей. Схема обучения остаётся знакомой.

БУСТИНГ

$$\hat{y}(x) = \hat{y}^{(0)} + \eta \sum_{m=1}^{M} h_m(x)$$

Сумма поправок от деревьев.

Обучаем: условия в узлах и ответы в листьях. Режет плоскость на прямоугольники.

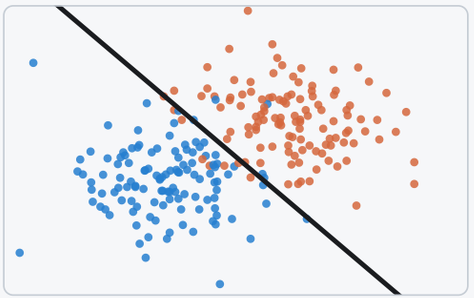


ЛИНЕЙНАЯ МОДЕЛЬ

$$\hat{y}(x) = w_1 x_1 + w_2 x_2 + \cdots + w_d x_d + b$$

Сумма вкладов признаков.

Обучаем: веса (коэффициенты) $w_1, \ldots, w_d$ и свободный член b. Делит плоскость одной прямой.



Данные
train, validation, test

→

Прогноз
модель с параметрами

→

Ошибка
функция потерь

→

Обновление
меняем параметры

Четыре разных решения в одной задаче

Что выбираем	Вопрос	Пример сегодня
Модель	Как из признаков получается прогноз?	$\hat{y} = w \cdot x + b$
Функция потерь	Что минимизируем при обучении?	средний квадрат ошибки, MSE
Способ оптимизации	Как ищем минимум?	формула МНК или градиентный спуск
Метрика	Как оцениваем на отложенных данных?	RMSE на validation

Датасет Auto MPG: экономичность автомобилей

398 автомобилей 1970–1982 годов. Задача: по характеристикам предсказать, сколько миль машина проезжает на галлоне топлива.

Столбец	Смысл
`mpg`	ответ: миль на галлон, больше — экономичнее
`weight`	масса, фунты
`horsepower`	мощность, л. с.; у 6 машин пропущена
`displacement`, `cylinders`	объём двигателя и число цилиндров
`acceleration`, `year`	время разгона и год модели
`origin`	категория: США, Европа, Япония

Разбиение

238 train — обучение.

80 validation — выбор настроек.

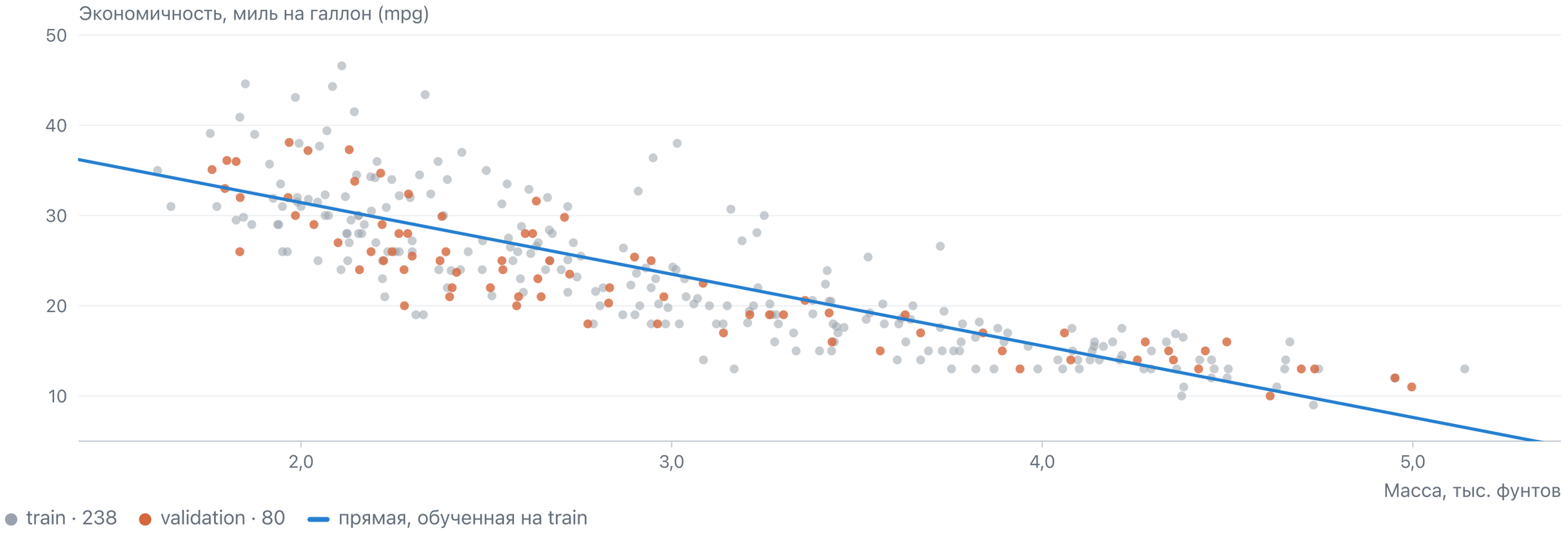
80 test — итоговая оценка, сегодня не трогаем.

Случайное разбиение 60 / 20 / 20, `random_state=42`.

Данные: UCI Machine Learning Repository · Auto MPG · R. Quinlan, 1993 · CC BY 4.0.

Один признак: прямая «масса → экономичность»

$$\hat{y} = 47{,}32 - 7{,}94 \cdot x \qquad x \text{ — масса в тысячах фунтов}, \quad \hat{y} \text{ — прогноз mpg}$$



RMSE на validation: **3,77 mpg**. Если всем предсказывать среднее по train, RMSE будет **7,08 mpg**.

Что означают наклон и свободный член

Наклон $w = -7{,}94$

Машина тяжелее на 1000 фунтов → прогноз меньше на 7,94 mpg.

Масса	Расчёт	Прогноз
3000 фунтов	47,32 – 7,94 · 3	23,51 mpg
4000 фунтов	47,32 – 7,94 · 4	15,57 mpg

У веса есть единицы: mpg на тысячу фунтов.

Свободный член $b = 47{,}32$

Прогноз при $x = 0$. Машин с нулевой массой нет: самая лёгкая в train весит 1613 фунтов.

Число b задаёт положение прямой. Отдельного смысла «экономичность невесомой машины» у него нет.

> Вес описывает **прогноз модели**. Он не доказывает, что облегчение конкретной машины даст именно такой прирост.

Единицы измерения меняют вес, но не прогноз

Запишем массу в фунтах вместо тысяч фунтов и обучим модель заново.

Единица массы	Вес w	Вклад для машины 3000 фунтов	Прогноз
тысячи фунтов	−7,939	$-7{,}939 \cdot 3 = -23{,}82$	23,51 mpg
фунты	−0,007939	$-0{,}007939 \cdot 3000 = -23{,}82$	23,51 mpg

Признак стал в 1000 раз больше — вес стал в 1000 раз меньше. Произведение то же.

По модулю веса нельзя сравнивать важность признаков в разных единицах.
К этому вернёмся, когда дойдём до стандартизации.

Несколько признаков: прогноз — сумма вкладов

$$\hat{y} = w_1 x_1 + w_2 x_2 + \cdots + w_d x_d + b = w \cdot x + b$$

Модель на трёх признаках. Одна машина из validation: 3360 фунтов, 110 л. с., 1979 год.

Признак	Значение x	Вес w	Вклад w · x
Масса, тыс. фунтов	3,36	−6,41	−21,53
Мощность, сотни л. с.	1,10	−1,14	−1,25
Год модели минус 1970	9	+0,78	+7,02
Свободный член b			+39,13
Прогноз			**23,37 mpg**

Настоящий ответ — 20,6 mpg. RMSE на validation снизился с 3,77 до 2,95 mpg. Модель обучена на 233 машинах train с известной мощностью.

Вся выборка одной формулой

$$\hat{y} = Xw + b$$

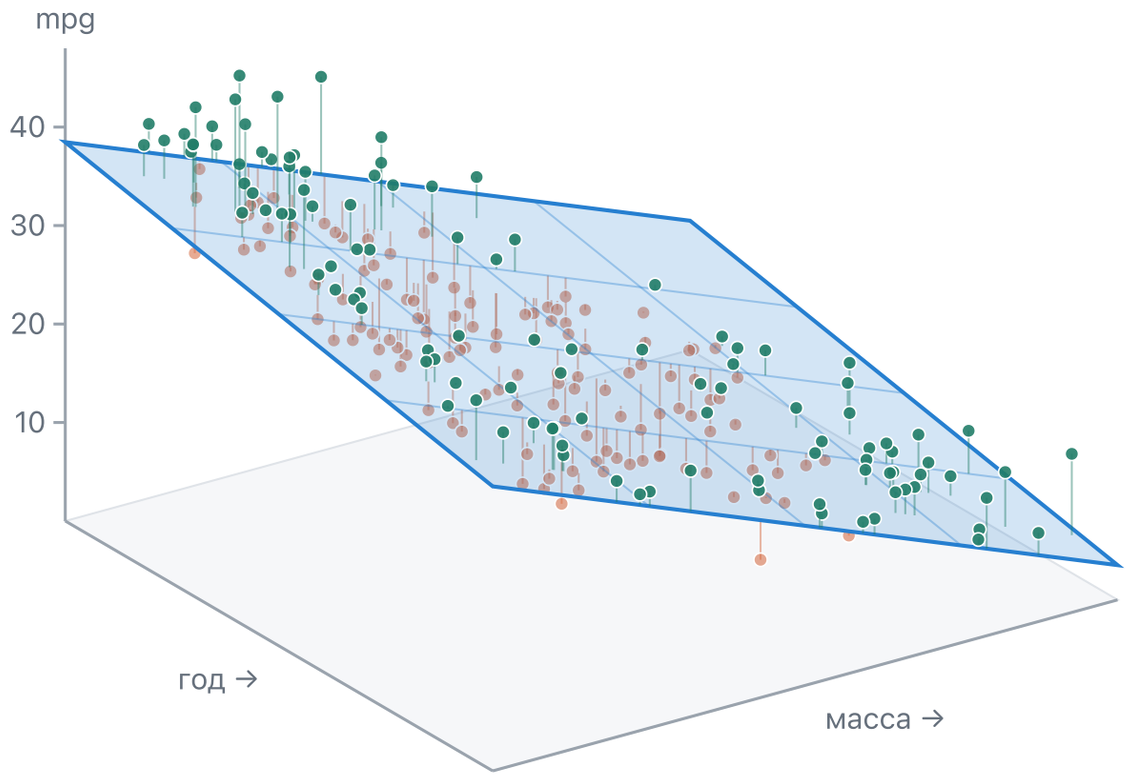
Обозначение	Размер	Смысл
X	$n \times d$	строка — объект, столбец — признак
w	d	по одному весу на признак
b	число	прибавляется к каждому прогнозу
$\hat{y}$	n	прогнозы для всех объектов

```
w = np.array([-6.41, -1.14, 0.78])
y_pred = X @ w + b   # X: (233, 3) → (233,)
```

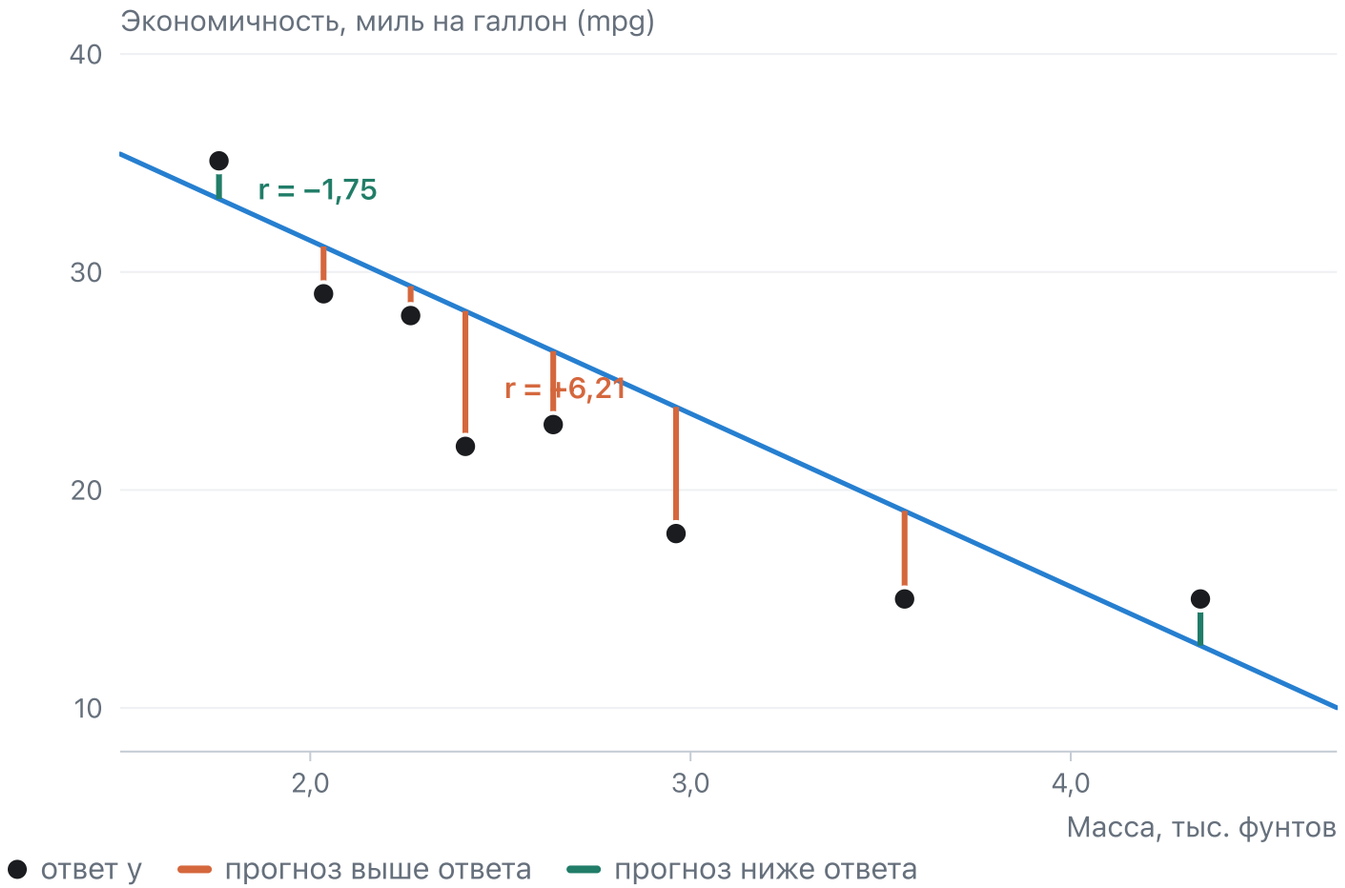
Оператор `@` из лекции по NumPy: каждая строка X скалярно умножается на w.

Один признак — прямая, **два — плоскость**, больше — гиперплоскость. Масса и год → mpg:

Повернуть 34° ● над плоскостью ● под ней



Остаток — на сколько прогноз разошёлся с ответом



$$r_i = \hat{y}_i - y_i$$

r > 0 — модель завысила прогноз.

r < 0 — занизила.

Машина	y	ŷ	r
1760 фунтов	35,1	33,35	−1,75
2408 фунтов	22,0	28,21	+6,21

На графике восемь машин из validation. Сегодня остаток всегда «прогноз минус ответ».

Откуда берутся веса? Начнём с самой простой модели

До сих пор веса были готовыми. Чтобы их подобрать, нужно решить, что считать ошибкой — выбрать функцию потерь. Посмотрим на модель без признаков: $\hat{y} = c$, одна константа для всех.



c = **10,0** · средний квадрат ошибки **66,0** (минимум 50,0 при c = 6, это среднее) · средний модуль **8,0** (минимум 4,2 при c = 3, это медиана)

Разные функции потерь — разные лучшие константы: квадраты ошибок (MSE) выбирают **среднее**, модули (MAE) — **медиану**. Выброс 20 тянет среднее сильнее.

Метод наименьших квадратов

Ищем веса, при которых средний квадрат остатка на train минимален.

$$J(w, b) = \frac{1}{2n} \sum_{i=1}^{n} \underbrace{\left(w\, x_i + b - y_i\right)}_{r_i}^2 \longrightarrow \min_{w,\, b}$$

Почему квадрат
Знак ошибки не важен. Большие промахи штрафуются сильнее.

Зачем 1/2
Сократится с двойкой при взятии производной. Положение минимума не меняет.

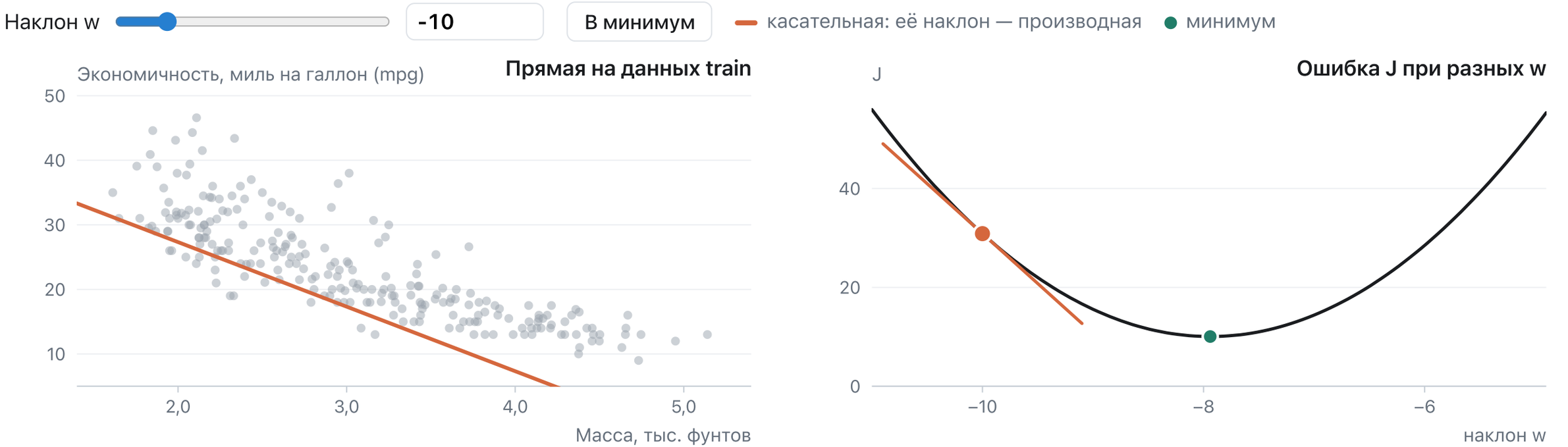
Связь с метриками
$J = \frac{1}{2}\,\mathrm{MSE}$, $\mathrm{RMSE} = \sqrt{\mathrm{MSE}}$. Минимум у всех трёх — при одних и тех же весах.

```
from sklearn.linear_model import LinearRegression
model = LinearRegression().fit(X_train, y_train)
model.coef_, model.intercept_   # [-7.94], 47.32
model.predict(X_val)
```

В sklearn МНК готов: `LinearRegression` сам подбирает веса, при которых прямая ближе всего к точкам. Дальше — как он это делает.

Минимум ищем через производную

Зафиксируем b = 47,32 и будем менять только наклон w. Ошибка J по w — парабола.



w = **−10,00** · J = **30,87** · производная dJ/dw = **−20,20** — меньше нуля: J уменьшится, если увеличить w. Минимум: w = −7,94

В минимуме производная равна нулю. Слева от него она отрицательна, справа — положительна: знак подсказывает, куда двигать вес.

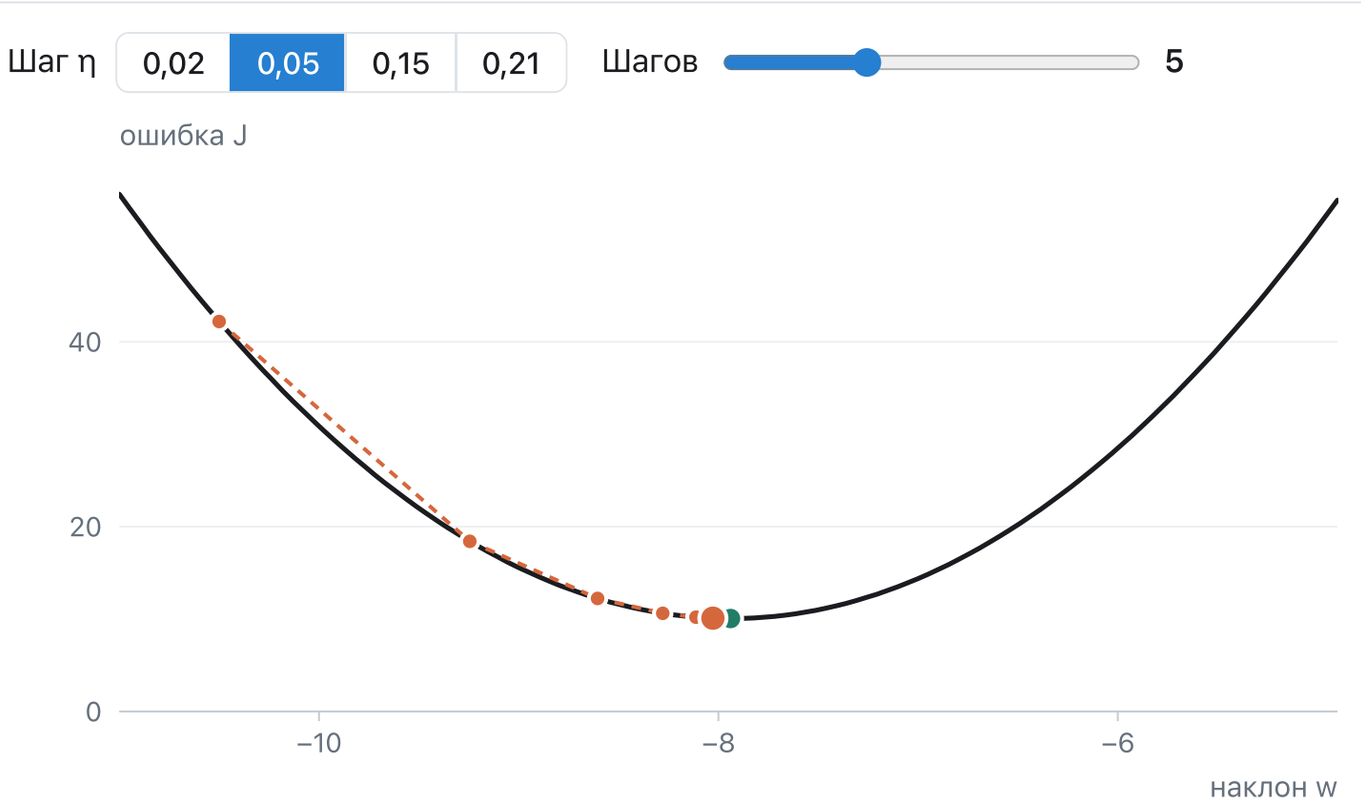
Градиентный спуск: шагаем против производной

$$w \leftarrow w - \eta \cdot \frac{dJ}{dw}$$

Производная отрицательна — вес увеличиваем, положительна — уменьшаем. η — размер шага, как `learning_rate` в бустинге.

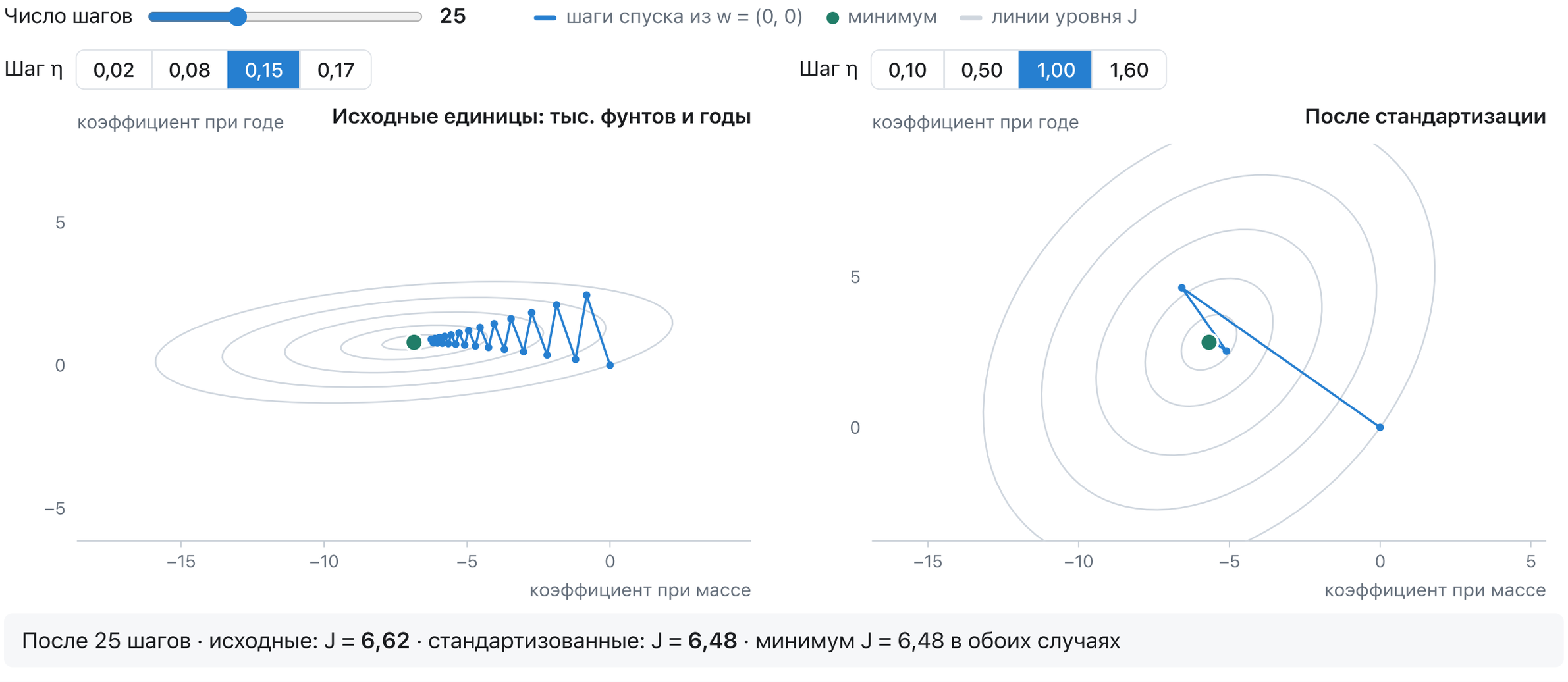
Весов много — каждый двигаем против своей производной. Производная по весу — среднее «остаток × признак».

```
for step in range(100):
    r = X @ w + b - y          # остатки
    w -= eta * X.T @ r / len(y)
    b -= eta * r.mean()
```



Старт w = −10,5 · после 5 шагов w = **−8,03**, J = **10,10** · минимум: w = −7,94, J = 10,06

Масштаб признаков меняет путь градиентного спуска



Auto MPG, train: предсказываем mpg по массе и году модели, признаки и ответ центрированы. По осям — коэффициенты (веса) при массе и при годе. Масштабы осей на каждом графике равны.

МНК — цель, градиентный спуск — способ

ТОЧНОЕ РЕШЕНИЕ

`LinearRegression`

Находит минимум сразу, без шагов: решает систему «все производные равны нулю».

Нет шага и числа итераций. Масштаб признаков на прогнозы не влияет.

Подходит, пока таблица помещается в память.

ИТЕРАЦИИ

Градиентный спуск, `SGDRegressor`

Двигается к минимуму небольшими шагами.

Нужны шаг η и стандартизация признаков.

Работает на очень больших данных и с любой дифференцируемой потерей — так обучается логрег и нейросети.

Если спуск сошёлся, прогнозы совпадают с точным решением: минимизируем одну и ту же функцию J.

БЫСТРЫЙ ОПРОС

Массу перевели из тысяч фунтов в килограммы и заново обучили МНК на том же train. Что изменится?

A Прогнозы и RMSE на validation

B Ничего: вес останется −7,94

✓ Вес при массе изменится, прогнозы останутся теми же

D Вес при массе останется прежним, изменится только свободный член b

Умножение признака на константу МНК компенсирует весом: w · x не меняется. У точного решения без штрафа прогнозы от единиц измерения не зависят.

БЫСТРЫЙ ОПРОС

Градиентный спуск для линейной регрессии расходится: J растёт с каждым шагом. Что пробуем первым?

A Добавить больше итераций

✓ Уменьшить шаг η и стандартизовать признаки

C Заменить MSE на RMSE

D Добавить к функции потерь Ridge-штраф

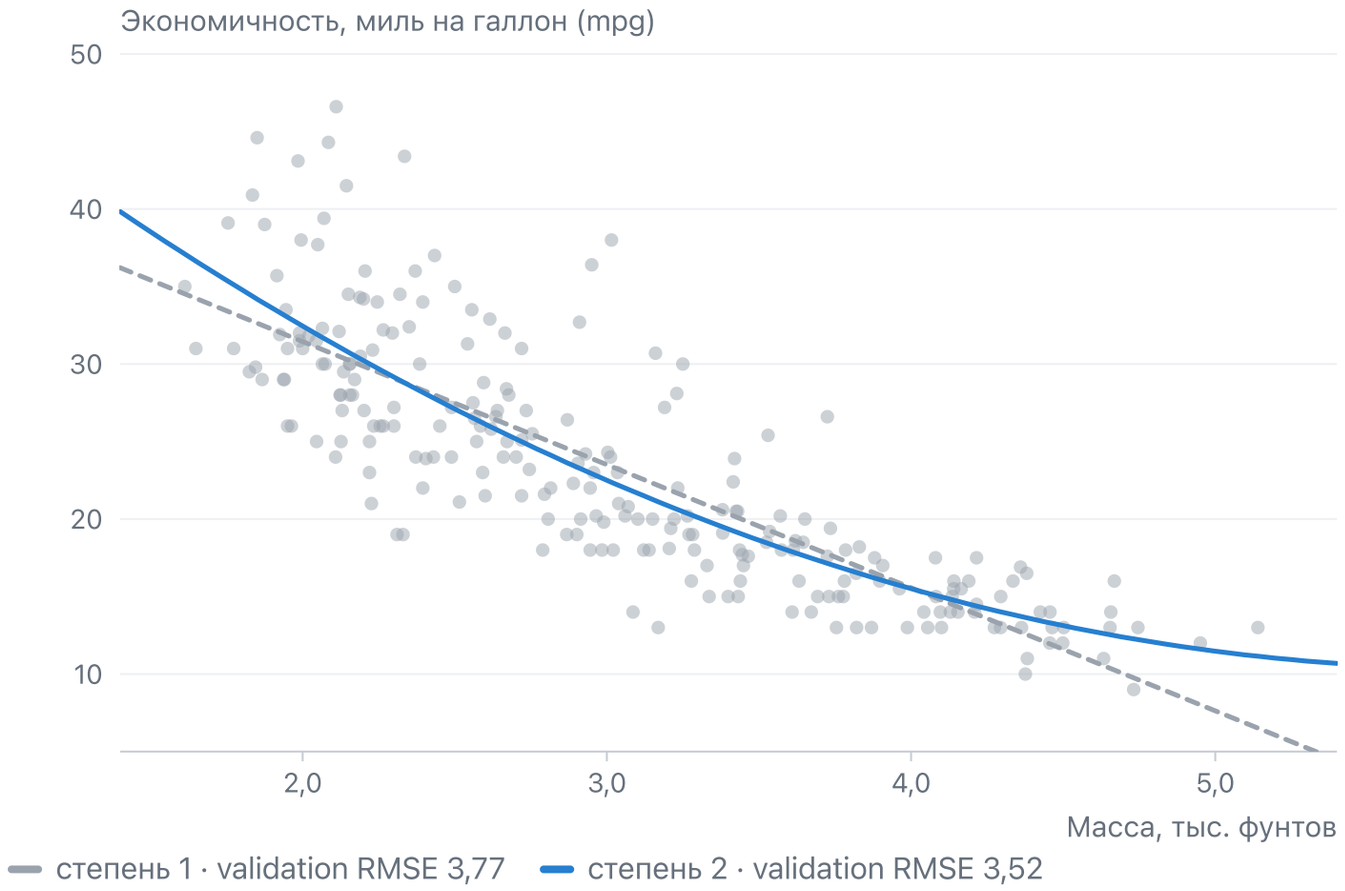
Расходимость — признак слишком большого шага для самого «крутого» направления. Стандартизация выравнивает масштабы, после неё один шаг подходит всем весам. Штраф делает функцию потерь только круче — расходимость от него не исчезнет.

ЧАСТЬ 2

Признаки, масштаб и штраф

Линейная модель проста. Её возможности определяют признаки, которые мы ей дали, и ограничения на веса.

Линейная модель умеет рисовать кривые



Добавим в таблицу новый столбец — квадрат массы:

$$\hat{y} = 61{,}22 - 17{,}34\,x + 1{,}48\,x^2$$

График — парабола. Но модель по-прежнему **линейна по весам**: прогноз — сумма «вес × столбец».

Алгоритм обучения тот же — МНК. Изменилась только матрица признаков: было $[x]$, стало $[x,\ x^2]$.

«Линейная» — про то, как в формулу входят веса, а не про форму графика.

Какие признаки можно добавить

$$x \longrightarrow \varphi(x) \longrightarrow \hat{y} = w \cdot \varphi(x) + b$$

Степени и функции

$x^2,\ \sqrt{x},\ \ln x$

Изгиб зависимости. Логарифм — только для положительных значений.

Взаимодействия

$x_1 \cdot x_2$

Эффект одного признака зависит от другого. Без такого столбца вклады просто складываются.

Категории

one-hot: столбцы из 0 и 1

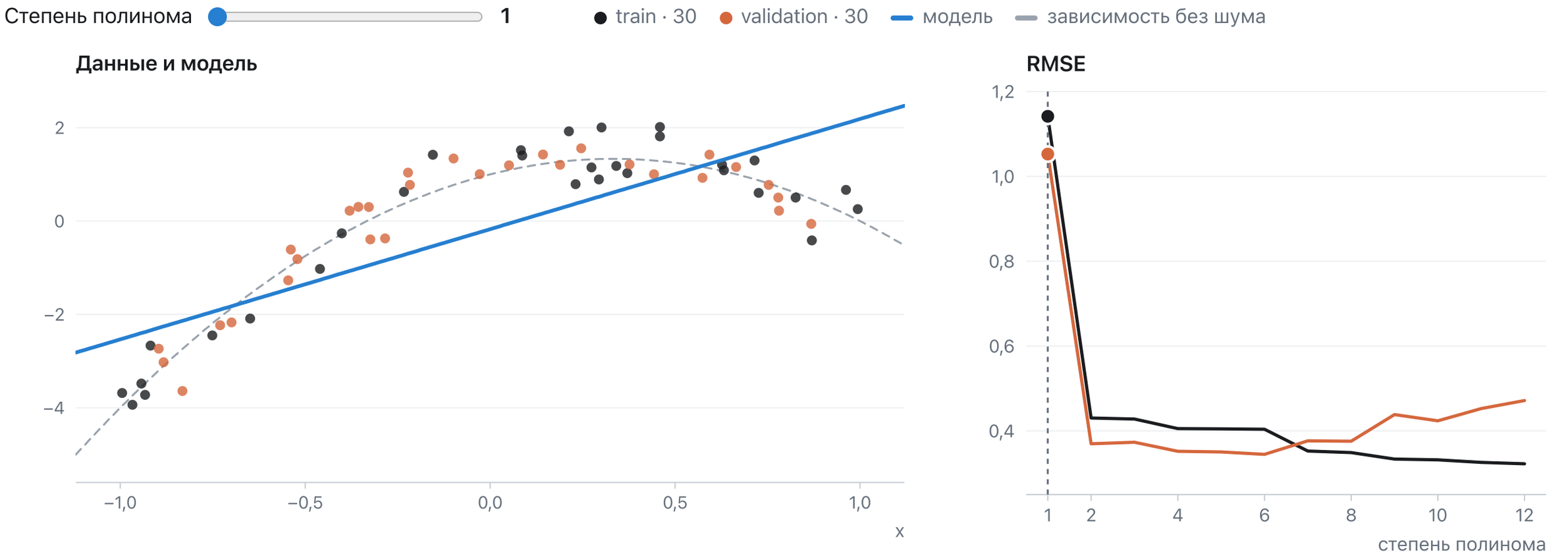
У каждой категории свой сдвиг прогноза.

Пример one-hot. Происхождение машины: США — опорная категория, для Европы и Японии — по столбцу.

$$\hat{y} = 43{,}64 - 7{,}03 \cdot \text{масса} + 2{,}41 \cdot [\text{Европа}] + 2{,}85 \cdot [\text{Япония}]$$

При той же массе прогноз для японской машины на 2,85 mpg выше, чем для американской. На validation RMSE почти не изменился: 3,78 против 3,77.

Больше признаков — больше возможностей и больше риска



Степень **1** · признаков: 1 · RMSE train **1,142** · RMSE validation **1,053** · минимум validation: степень 6 (0,345), у степени 2 — 0,370

Учебная синтетика: $y = 1 + 2x - 3x^2 +$ шум $N(0; 0{,}4^2)$, $x \sim U(-1; 1)$, генератор NumPy с seed = 0. Модель обучается только на train.

Нормализация и стандартизация: один масштаб для всех признаков

Меняем масштаб каждого признака. Порядок объектов по признаку остаётся прежним.

Нормализация · `MinMaxScaler`

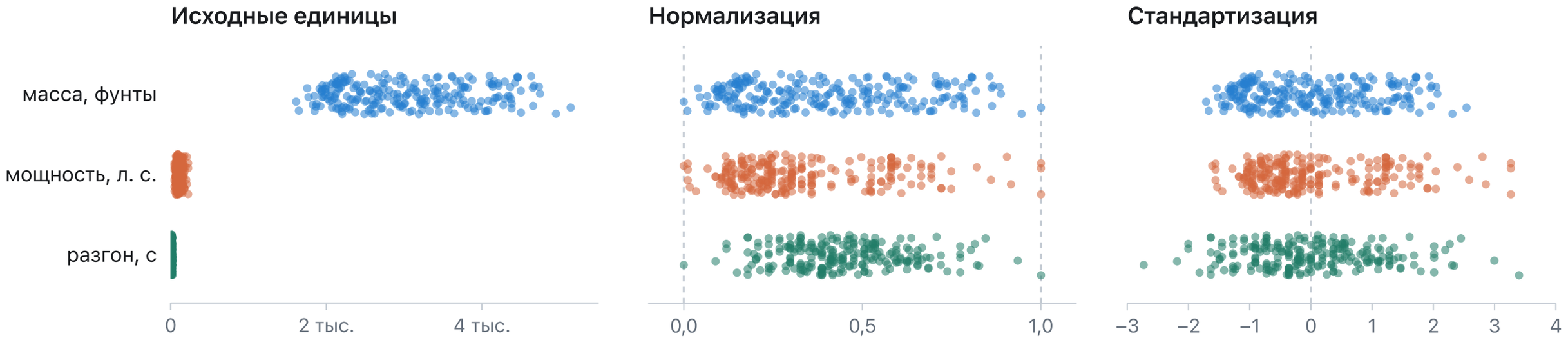
$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \in [0;\ 1]$$

Один выброс сожмёт остальные значения к краю.

Стандартизация · `StandardScaler`

$$x' = \frac{x - \bar{x}}{s_x}: \quad \text{среднее } 0,\ \text{разброс } 1$$

Основной выбор для линейных моделей.



Auto MPG, 233 машины train с известной мощностью. На общей оси в исходных единицах мощность и разгон почти не видны: их числа в десятки раз меньше массы. Машина 3360 фунтов: нормализация (3,36 – 1,61) / (5,14 – 1,61) = 0,50, стандартизация (3,36 – 3,02) / 0,83 = 0,41.

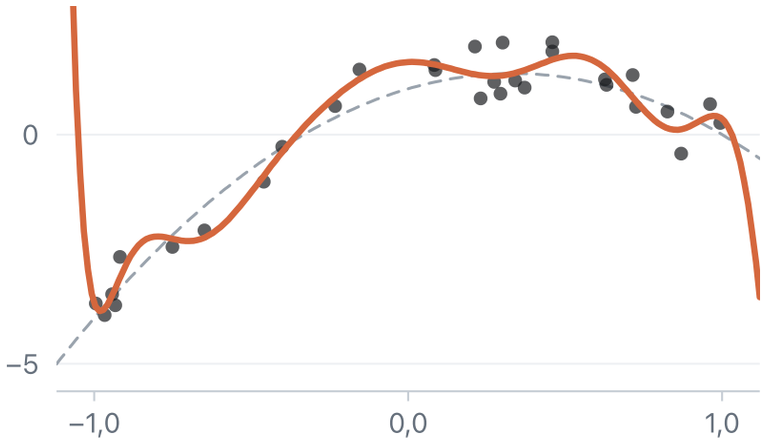
Регуляризация: штраф за большие веса

$$J(w, b) + \lambda \sum_j w_j^2$$

Меняем не данные, а **функцию потерь**: большие веса теперь стоят дорого.

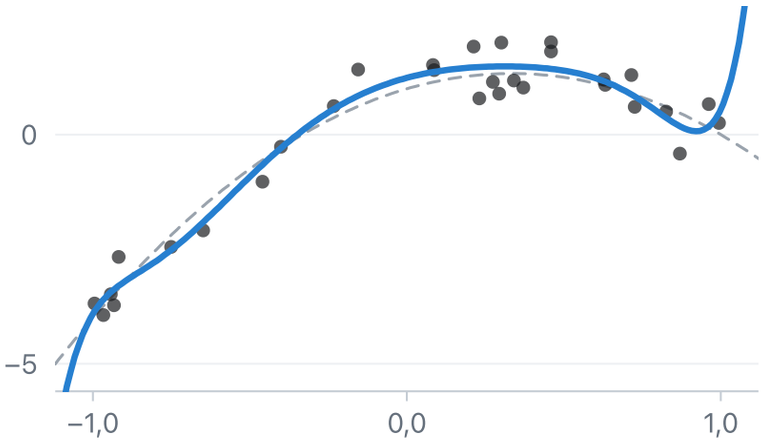
```
model = LinearRegression()              # было
model = make_pipeline(StandardScaler(),
                      Ridge(alpha=1.0))  # стало
model.fit(X_train, y_train)
```

Без штрафа



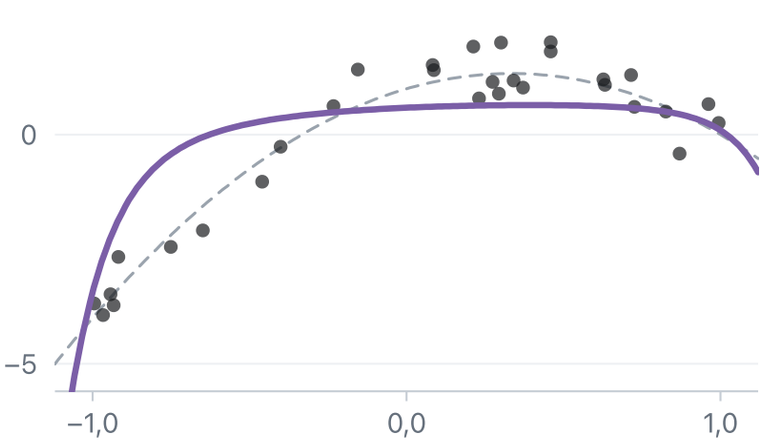
‖w‖ = **275** · RMSE validation **0,47**

Подобранный штраф, alpha = 0,03



‖w‖ = **3,1** · RMSE validation **0,33**

Сильный штраф, alpha = 100



‖w‖ = **0,4** · RMSE validation **0,98**

Полином степени 12 из опыта со степенями. Чёрные точки — train, пунктир — зависимость без шума. Степени x, …, x^{12} стандартизованы по train; подобранный alpha — лучший по validation из 15 значений от 0,0001 до 1000.

LinearRegression, Ridge, Lasso: одна модель, разные штрафы

У всех трёх прогноз один и тот же — $\hat{y} = w \cdot x + b$, и тот же код: `fit`, `predict`, `coef_`. Отличается только то, что минимизируем.

БЕЗ ШТРАФА

`LinearRegression()`

$$\text{MSE}$$

Веса — какие получились.

Признаков мало, объектов много, признаки не дублируют друг друга.

L2-ШТРАФ

`Ridge(alpha=…)`

$$\text{MSE} + \lambda \sum_j w_j^2$$

Все веса меньше, но не ноль. Похожие признаки делят вклад.

Выбор по умолчанию.

L1-ШТРАФ

`Lasso(alpha=…)`

$$\text{MSE} + \lambda \sum_j |w_j|$$

Часть весов — ровно ноль: признак выпадает.

Признаков много, полезны немногие.

```
LinearRegression().fit(X_train, y_train)
Ridge(alpha=10).fit(X_train, y_train)
Lasso(alpha=0.7).fit(X_train, y_train)
```

Сила штрафа λ в sklearn — `alpha`. При `alpha = 0` Ridge и Lasso совпадают с МНК, при большом — все веса около нуля. Свободный член b не штрафуется. Признаки перед штрафом стандартизуем.

Когда масштаб признаков важен

Модель или метод	Нужно масштабировать?	Почему
Дерево, случайный лес, бустинг	**нет**	условие «x ≤ порог» не зависит от единиц
МНК без штрафа, точное решение	**нет**	вес компенсирует единицы, прогнозы те же
Градиентный спуск	**да**	один шаг η на все веса
Ridge, Lasso, логрег со штрафом	**да**	штраф сравнивает веса между собой
kNN, k-means	**да**	расстояние складывает разности признаков

```
model = make_pipeline(
    StandardScaler(),
    Ridge(alpha=10),
)
model.fit(X_train, y_train)
model.predict(X_val)
```

Среднее и разброс считаем **только по train**. `Pipeline` делает это сам: при `fit` обучает scaler, при `predict` применяет готовый.

Похожие признаки: МНК не знает, как поделить вклад

Пусть признак x_2 — точная копия x_1.

Данные задают только сумму весов:

$$w_1 + w_2 = 2$$

Как её поделить — модели всё равно: подходят и 1 + 1, и 102 – 100.

Поэтому веса случайны и могут стать огромными.

Три модели на одних данных:

x_1	x_2	y	$2x_1 + 0x_2$	$x_1 + x_2$	$102x_1 – 100x_2$
1	1	2	2	2	2
2	2	4	4	4	4
3	3	6	6	6	6
2	2,1	?	4	4,1	**–6**

На train все три безошибочны. Но пусть у нового объекта x_2 чуть отличается от x_1 — модель с огромными весами усиливает разницу в 100 раз.

Есть похожие признаки — по отдельному весу о важности не судим. Помогает Ridge (делит вклад поровну), Lasso (оставит один из группы) или удаление дубля.

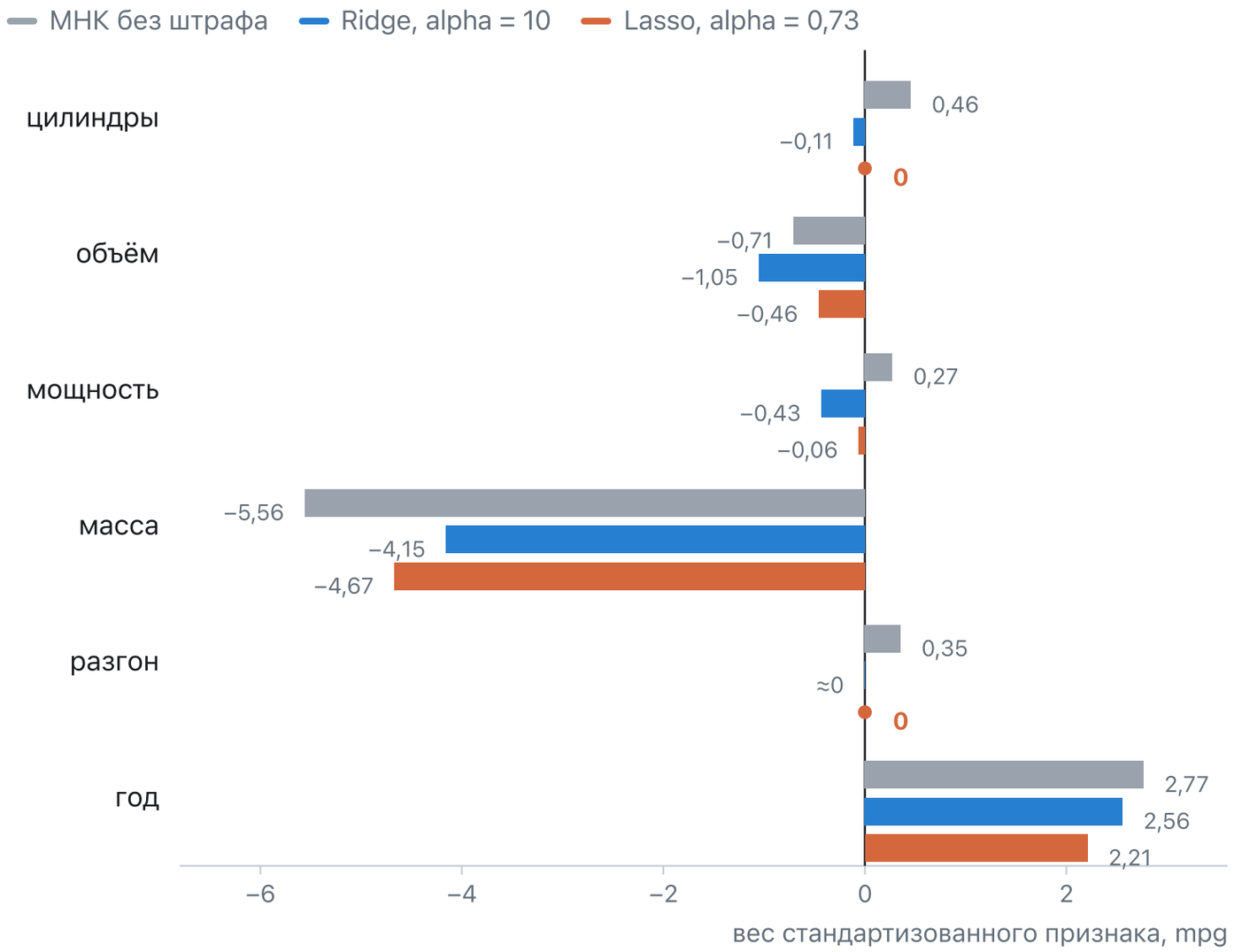
Когда включать штраф: сигналы

Сигнал	Как заметить
Модель переобучилась	ошибка на train намного меньше, чем на validation
Признаков много относительно объектов	`X_train.shape`: сотни столбцов на тысячу строк; полиномы, one-hot
Есть похожие признаки	`X_train.corr()`: пары с корреляцией около ±1
Веса огромные или скачут	`model.coef_` меняется от небольшого изменения train

```
for alpha in [0.01, 0.1, 1, 10, 100]:
    m = make_pipeline(StandardScaler(), Ridge(alpha=alpha))
    m.fit(X_train, y_train)
    print(alpha, mean_squared_error(y_val, m.predict(X_val)) ** 0.5)
```

Нет ни одного сигнала — штраф почти ничего не изменит, можно без него. Решаем не на глаз: сравниваем с `LinearRegression` на validation.

Ridge и Lasso на Auto MPG: что стало с весами



Шесть признаков, все стандартизованы. Цилиндры, объём, мощность и масса сильно связаны.

МНК. Веса цилиндров, мощности и разгона положительные. По отдельности их не прочитать: модель делит вклад между похожими признаками.

Ridge. Общая величина весов меньше: ‖w‖ 6,28 → 5,01. Вклад похожих признаков перераспределился: объём −0,71 → −1,05, мощность +0,27 → −0,43. Нулей нет.

Lasso. Цилиндры и разгон — **ровно ноль**. Остались масса, объём, год и небольшой вес мощности.

RMSE на validation: МНК 3,02 · Ridge 3,01 · Lasso 2,79 mpg. alpha у каждой модели выбран по validation.

Силу штрафа выбираем по validation

1. Разбить данные до любой подгонки.
2. Для каждого значения из сетки обучить `Pipeline` на train.
3. Выбрать значение с лучшей метрикой на validation.
4. Test — один раз, в самом конце.



При маленьком alpha штраф почти не действует: RMSE как у МНК без штрафа (3,02). Auto MPG, 6 признаков.

Как λ называется в scikit-learn

Класс	Параметр	Больше значение —
`Ridge`	`alpha`	сильнее штраф
`Lasso`	`alpha`	сильнее штраф
`LogisticRegression`	`C`	**слабее штраф**

Классы по-разному нормируют ошибку, поэтому подходящий alpha у Ridge и у Lasso разный — подбираем для каждого.

БЫСТРЫЙ ОПРОС

Как правильно применить StandardScaler?

A Обучить на всей таблице, потом разбить на train и validation

✓ Обучить на train, применить к train, validation и test

C Обучить отдельно на train, отдельно на validation и отдельно на test

D Не масштабировать: Ridge сам подстроит веса под единицы измерения

Среднее и разброс — тоже параметры, выученные по данным. Если считать их по всей таблице, информация из validation и test утекает в обучение. Pipeline исключает эту ошибку.

БЫСТРЫЙ ОПРОС

Lasso обнулил вес признака «число цилиндров» и оставил «массу». Какой вывод верен?

A Число цилиндров не связано с экономичностью

B Масса — причина расхода, цилиндры — нет

C Цилиндры можно выкинуть из таблицы для любой модели

✓ При этом штрафе модели хватило других признаков, похожих на этот

Цилиндры сильно коррелируют с массой и объёмом. Lasso оставляет один признак из группы похожих; какой именно — может зависеть от выборки. О причинности веса не говорят.

ЧАСТЬ 3

Логистическая регрессия

Несмотря на название, это классификатор. От линейной регрессии остаётся score $w \cdot x + b$, меняется то, что мы с ним делаем.

Что меняется при переходе к классификации

Два класса: y = 0 или y = 1. Хотим не только метку, но и уверенность.

	Линейная регрессия	Логистическая регрессия
Что считаем	$\hat{y} = w \cdot x + b$	score $z = w \cdot x + b$
Что выдаём	число $\hat{y}$	вероятность $p = \sigma(z)$, затем класс по порогу
Что минимизируем	квадрат ошибки	log loss
Градиент по score	$\hat{y} - y$	$p - y$

ВОПРОС В ЗАЛ

Почему нельзя просто обучить линейную регрессию на метках 0 и 1 и назвать прогноз вероятностью?

Шансы: как перейти от вероятности к числу без границ

Вероятность

$p \in (0;\ 1)$

p = 0,8

→

Шансы

$\frac{p}{1-p} \in (0;\ \infty)$

0,8 / 0,2 = 4, «четыре к одному»

→

Логарифм шансов

$\ln \frac{p}{1-p} \in (-\infty;\ \infty)$

ln 4 ≈ 1,39

Логарифм шансов принимает любые значения — как линейное выражение. Логистическая регрессия приравнивает их:

$$\ln \frac{p}{1-p} = w \cdot x + b$$

p = 0,5 ↔ шансы 1 ↔ логарифм шансов 0. Вероятности ниже 0,5 дают отрицательный score.

Сигмоида: обратно от score к вероятности

Выразим p из равенства $\ln \frac{p}{1-p} = z$:

$$\frac{p}{1-p} = e^z$$

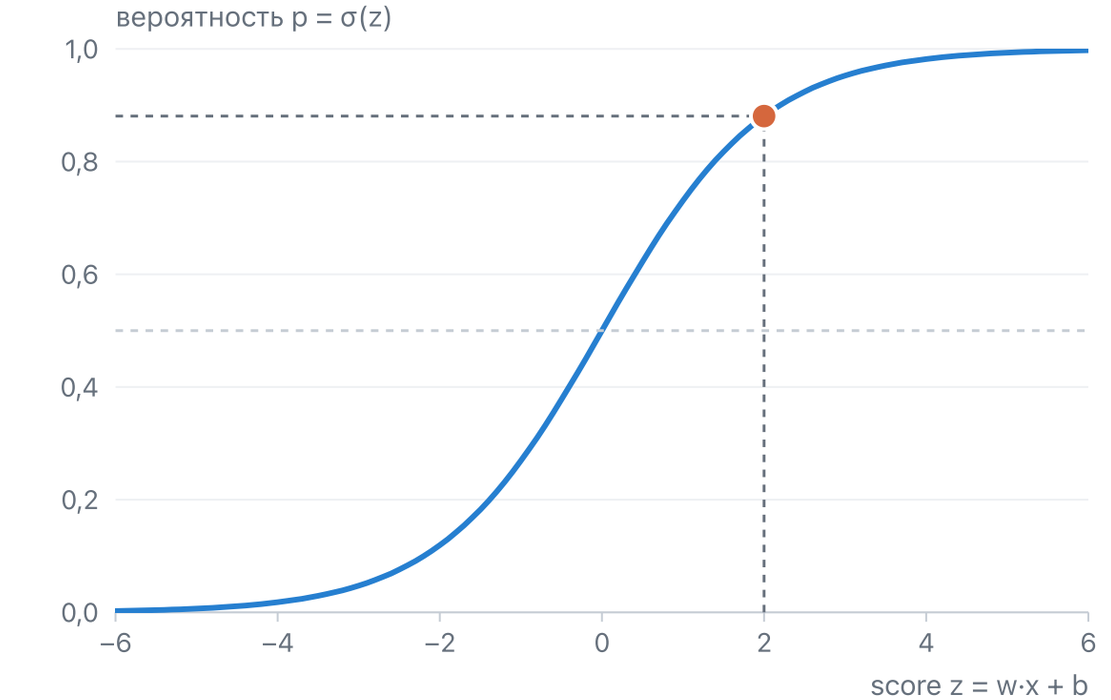
$$p = e^z - p\,e^z$$

$$p\,(1 + e^z) = e^z$$

$$p = \frac{e^z}{1+e^z} = \frac{1}{1+e^{-z}} = \sigma(z)$$

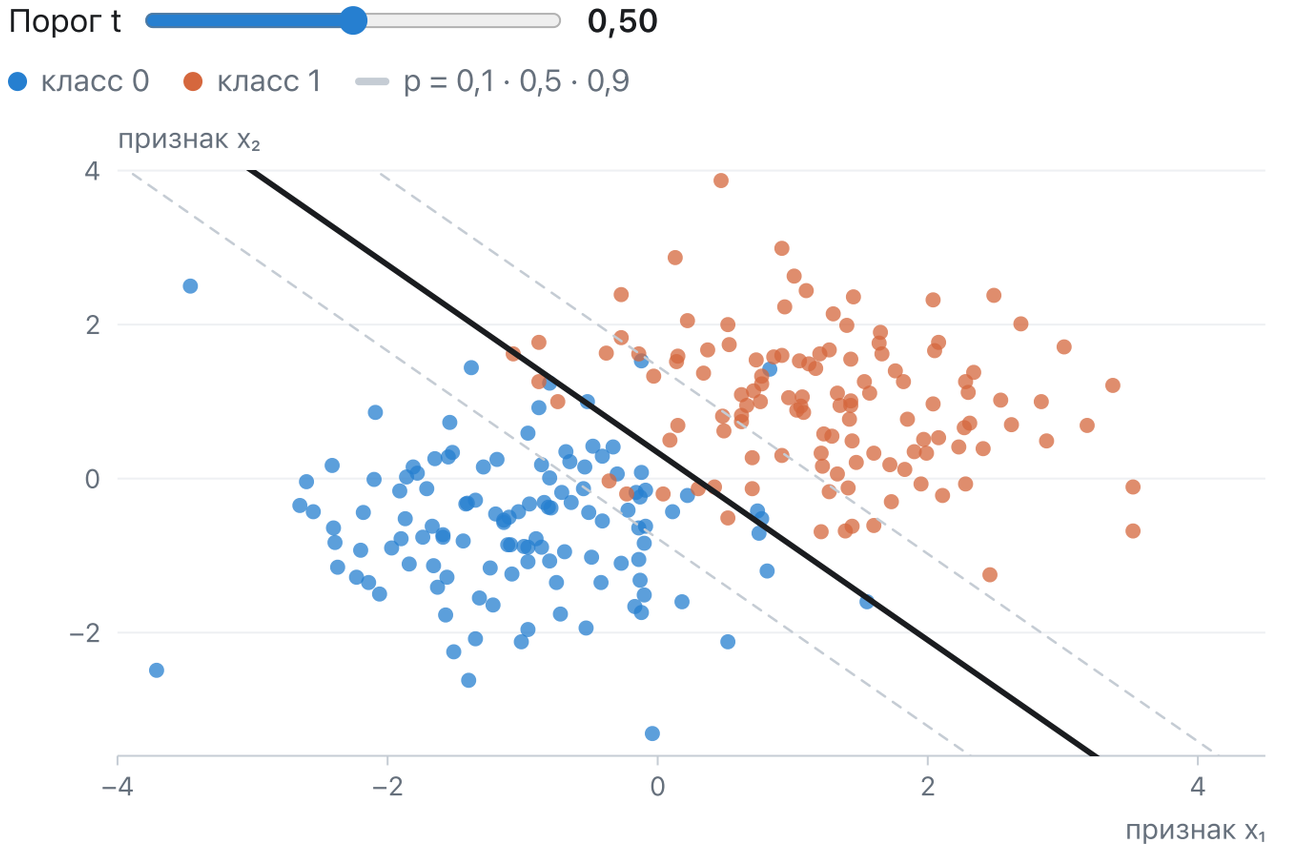
score z	−2	0	+2
p	0,12	0,50	0,88

Признак вырос на 1 → score вырос на w_j → шансы умножились на e^{w_j}. Вероятность при этом меняется не на постоянную величину.



z = **2,0** → p = **0,881** · шансы p/(1−p) = **7,39**

Порог превращает вероятность в класс



Порог t = **0,50** · граница: w·x + b = ln(t/(1−t)) = **0,00** · синих в классе 1: **5** · оранжевых в классе 0: **8**

Учебная синтетика: два нормальных облака по 120 точек, seed = 1; логрег обучен на показанных точках.

$$\hat{y} = \begin{cases} 1, & p \geq t \\ 0, & p < t \end{cases}$$

Условие $p \geq t$ — то же, что

$$w \cdot x + b \;\geq\; \ln \frac{t}{1-t}$$

Граница между классами — **прямая**. При $t = 0{,}5$ справа стоит 0.

Смена порога сдвигает границу параллельно. Веса модели при этом не меняются.

Как обучать логрег: штраф за каждый объект

Смотрим, какую вероятность модель дала правильному классу. Чем она меньше, тем больше штраф.

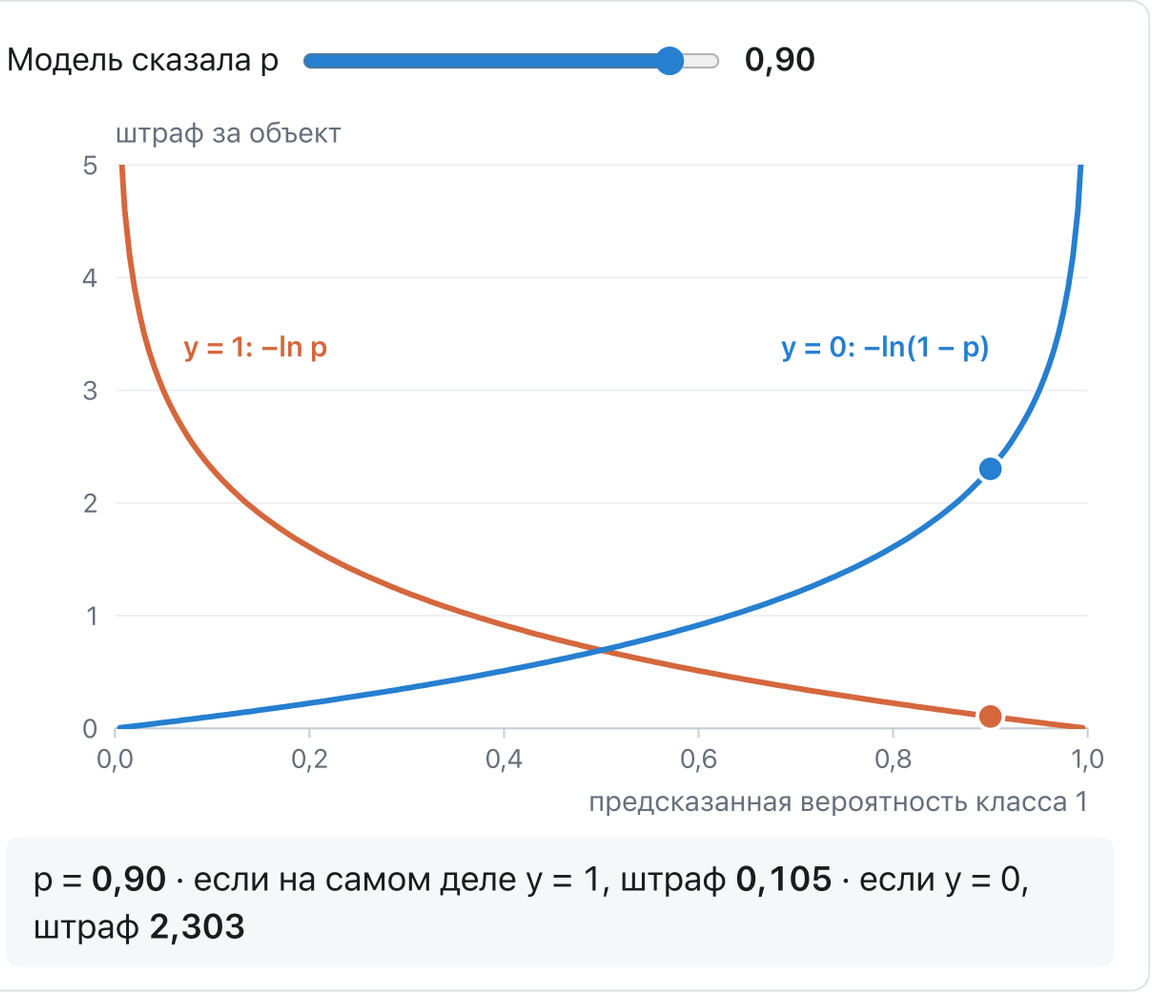
$$\text{штраф} = -\ln(\text{вероятность правильного класса})$$

Если ответ 1 — берём p, если 0 — берём $1 - p$. Средний штраф по train — функция потерь **log loss**, её и минимизируем.

$$J = -\frac{1}{n}\sum_{i=1}^{n}\Big[y_i \ln p_i + (1 - y_i)\ln(1 - p_i)\Big]$$

Ответ y	Модель: p	Вероятность правильного	Штраф
1	0,9	0,9	0,105
0	0,2	0,8	0,223
1	0,3	0,3	1,204
Log loss — средний штраф			**0,511**

Уверенная ошибка стоит дорого



p = **0,90** · если на самом деле y = 1, штраф **0,105** · если y = 0, штраф **2,303**

Штраф за один объект класса 1 равен $-\ln p$.

Модель сказала p	Штраф, если y = 1
0,99	0,010
0,90	0,105
0,50	0,693
0,10	2,303
0,01	4,605

«Не знаю» (p = 0,5) стоит 0,693. Уверенный промах (p = 0,01) — почти в семь раз дороже.

Log loss оценивает сами вероятности, а не только то, по какую сторону порога они оказались.

Обучение логрега: снова «прогноз минус ответ»

Минимум log loss формулой не найти — только градиентным спуском. Производная по score получается удивительно простой:

$$\frac{d\,\text{штраф}}{dz} = p - y$$

y	p	p – y	Что делает спуск
1	0,9	–0,1	чуть поднимает score
1	0,3	–0,7	сильно поднимает score
0	0,8	+0,8	сильно опускает score

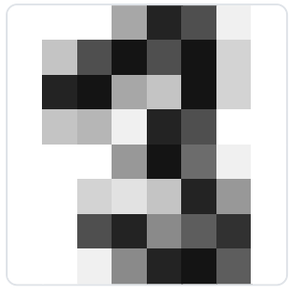
```
z = X @ w + b
p = 1 / (1 + np.exp(-z))      # единственное отличие
w -= eta * X.T @ (p - y) / len(y)
b -= eta * (p - y).mean()
```

У линейной регрессии на этом месте стоял остаток $\hat{y} - y$.

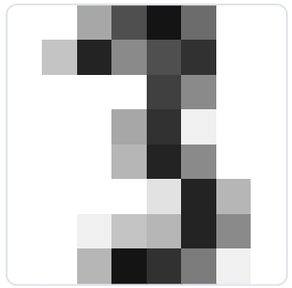
> В бустинге для классификации новое дерево обучали на $y - p$ — это тот же сигнал с обратным знаком.

Пример: отличить цифру 3 от цифры 8

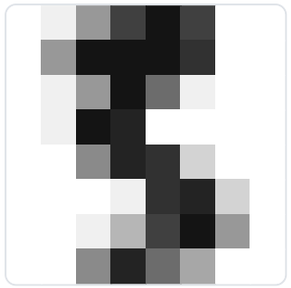
Датасет Digits: изображения 8×8, каждый пиксель — признак. Всего 64 признака, класс 1 — «восьмёрка».



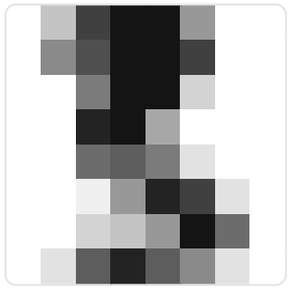
ответ: **3**
z = −2,8
p(8) = **0,058**
верно



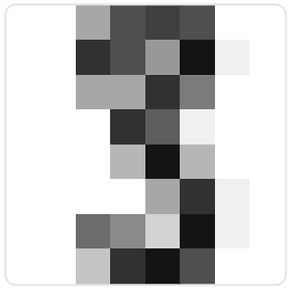
ответ: **3**
z = −13,2
p(8) = **0,000**
верно



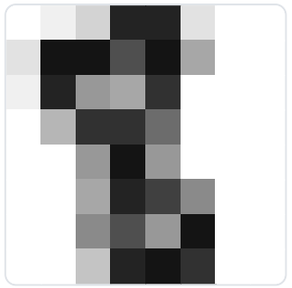
ответ: **3**
z = 0,8
p(8) = **0,693**
ошибка



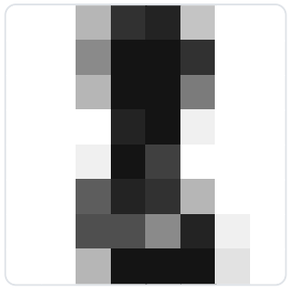
ответ: **3**
z = −1,1
p(8) = **0,258**
верно



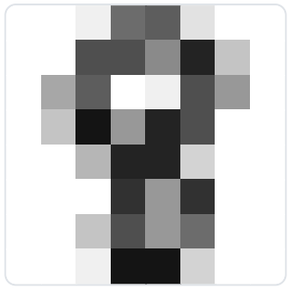
ответ: **3**
z = −1,1
p(8) = **0,250**
верно



ответ: **8**
z = 7,9
p(8) = **1,000**
верно



ответ: **8**
z = 5,4
p(8) = **0,995**
верно



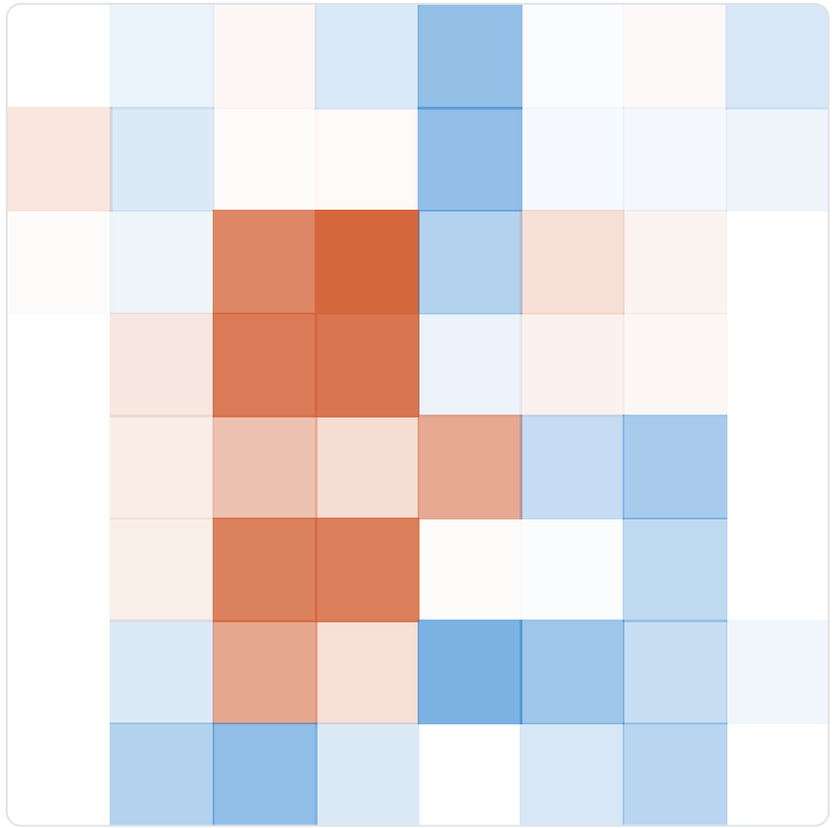
ответ: **8**
z = 8,4
p(8) = **1,000**
верно

357 изображений: **214 train · 71 validation · 72 test**, стратифицированное разбиение. Модель: `StandardScaler` + `LogisticRegression`. На рисунке — восемь изображений validation, решение при пороге 0,5.

Третья слева цифра: score 0,81 → p(8) = 0,693. Модель ошиблась, но и уверенность невысокая. У остальных семи ответ верный.

Данные: `sklearn.datasets.load_digits` — тестовая часть UCI Optical Recognition of Handwritten Digits.

Веса логрега можно нарисовать



- вес > 0: яркий пиксель повышает score «8»
- вес < 0: повышает score «3»

64 веса — по одному на пиксель. Разложим их в ту же сетку 8×8.

Оранжевые пиксели — в левой половине. У восьмёрки там левые стороны петель, у тройки пусто: если пиксель там ярче обычного, score «8» растёт.

$$z = b + \sum_{j=1}^{64} w_j \cdot \frac{\text{яркость}_j - \text{средняя}_j}{\text{разброс}_j}$$

Яркость каждого пикселя стандартизована по train, b — свободный член.

Логрегу тоже нужен штраф за большие веса

Если классы на train разделяются прямой, без штрафа модели выгодно бесконечно увеличивать веса: вероятности уходят к 0 и 1, log loss на train стремится к нулю.

Поэтому в `LogisticRegression` L2-штраф, как в Ridge, включён по умолчанию.

Его силу задаёт параметр `C` — **обратная** сила штрафа:

> больше `C` — слабее штраф, веса больше.
> меньше `C` — сильнее штраф, веса меньше.

По умолчанию `C = 1`. Подбираем по validation, как alpha у Ridge.

C	Величина весов ‖w‖	Log loss train	Log loss validation
0,01	0,69	0,203	0,210
0,1	1,57	0,060	0,084
1	2,93	0,013	0,043
10	4,71	0,002	0,033
100	6,64	0,0004	0,037
10 000	7,62	0,0002	0,038

Цифры 3 и 8. С ростом C веса растут, ошибка на train падает до нуля, а на validation лучшая — при C = 10.

Что оцениваем: вероятности или решения

Логрег выдаёт вероятности, а решение — класс — получается после порога. Оценивать можно и то, и другое.

ВЕРОЯТНОСТИ

Log loss ↓

Насколько уверенно модель ставит правильный класс. Наказывает уверенные ошибки.

На вход — `predict_proba`. От порога не зависит.

РЕШЕНИЯ ПОСЛЕ ПОРОГА

Precision, recall, F1 ↑

Сколько ошибок каждого вида среди меток 0 и 1.

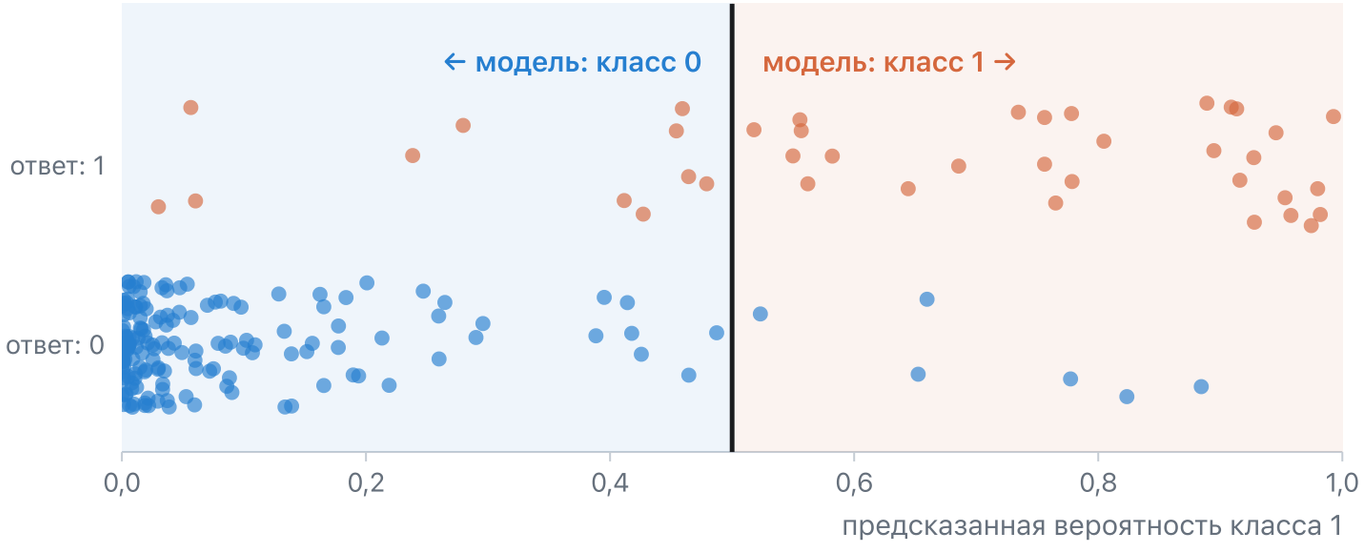
На вход — метки. Меняются при смене порога.

Цифры 3 и 8, validation: log loss 0,033; при пороге 0,5 F1 = 0,986, одна ошибка — тройка, принятая за восьмёрку.

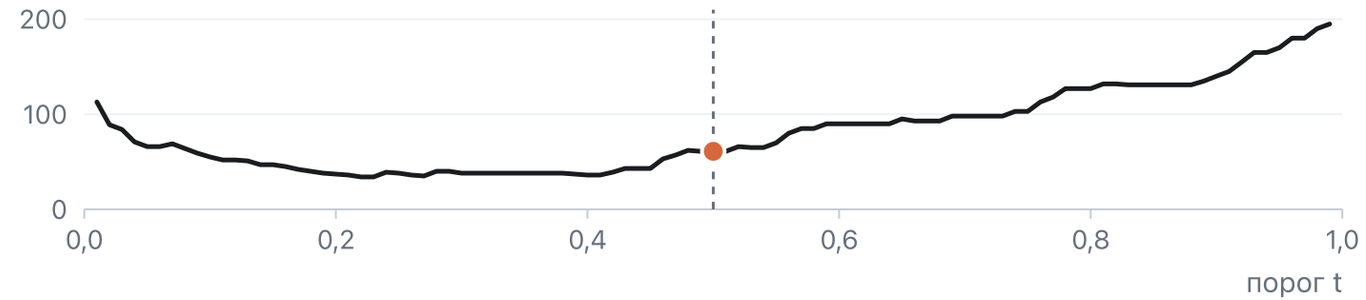
Хорошие вероятности ещё не значат хорошие решения при пороге 0,5. Порог выбирают отдельно.

Порог выбираем под задачу

Порог t: класс 1, если p ≥ t 0,50 | t = 0,5 | минимум стоимости: 0,23 | максимум F1: 0,42



Стоимость ошибок: FP + 5·FN



	модель: 1	модель: 0
ответ: 1	TP 29	FN 11
ответ: 0	FP 6	TN 154

Precision	0,829
Recall	0,725
F1	0,773
Стоимость FP + 5·FN	61

Учебная синтетика: два нормальных облака, доля класса 1 около 20%, логрег обучен на 600 объектах, показаны 200 объектов validation. При равной стоимости выбираем больший порог.

БЫСТРЫЙ ОПРОС

Логистическая регрессия выдала для объекта p = 0,3. Какой класс предскажет модель?

A Класс 0: вероятность меньше половины

B Класс 1 с вероятностью 0,3: модель выбирает класс случайно

✓ Зависит от порога: при t = 0,5 — класс 0, при t = 0,2 — класс 1

D Класс 0, а порог менять нельзя: 0,5 зашит в log loss

Вероятность и класс — разные величины. Метод predict использует порог 0,5, но это соглашение по умолчанию, а не свойство модели. Порог выбирают под цену ошибок.

БЫСТРЫЙ ОПРОС

У логрега вес признака «стаж» равен 0,7. Стаж вырос на 1 год, остальное не изменилось. Что произошло?

A Вероятность класса 1 выросла на 0,7

B Вероятность класса 1 выросла на 70%

✓ Score вырос на 0,7, шансы p/(1−p) умножились на $e^{0,7} \approx 2$

D Score вырос на 0,7, поэтому класс обязательно сменится

Вес линейно меняет score — логарифм шансов. Шансы умножаются на $e^{0,7} \approx 2,01$. Насколько при этом вырастет вероятность, зависит от того, где объект на сигмоиде: от 0,5 до 0,67, но от 0,95 только до 0,975.

БЫСТРЫЙ ОПРОС

В LogisticRegression увеличили C с 1 до 1000. Что произойдёт?

✓ Штраф ослабнет: веса вырастут, log loss на train уменьшится

B Штраф усилится: веса станут ближе к нулю

C Изменится порог классификации

D Ничего: C влияет только на скорость обучения

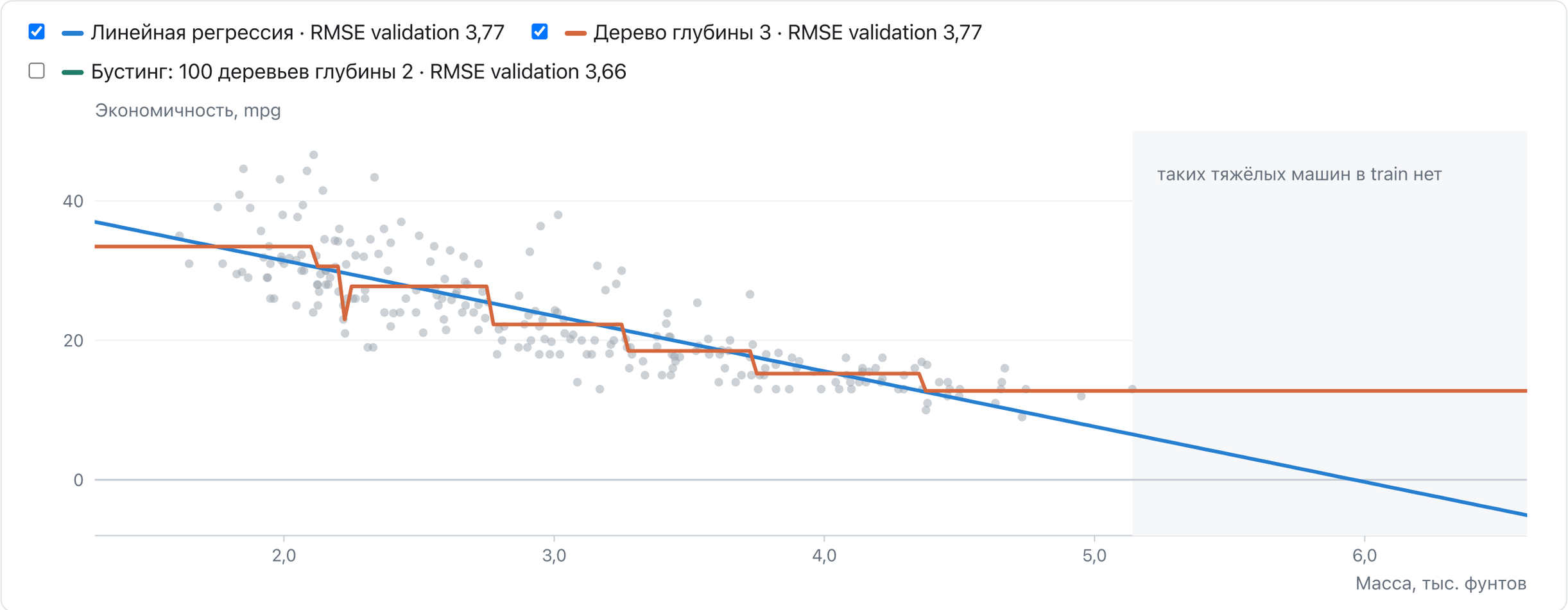
C — обратная сила регуляризации. Большое C разрешает большие веса. Качество на validation при этом может как вырасти, так и упасть — проверяем.

ЧАСТЬ 4

Собираем всё вместе

Деревья, бустинг, линейная и логистическая регрессия: чем они отличаются и с чего начинать новую задачу.

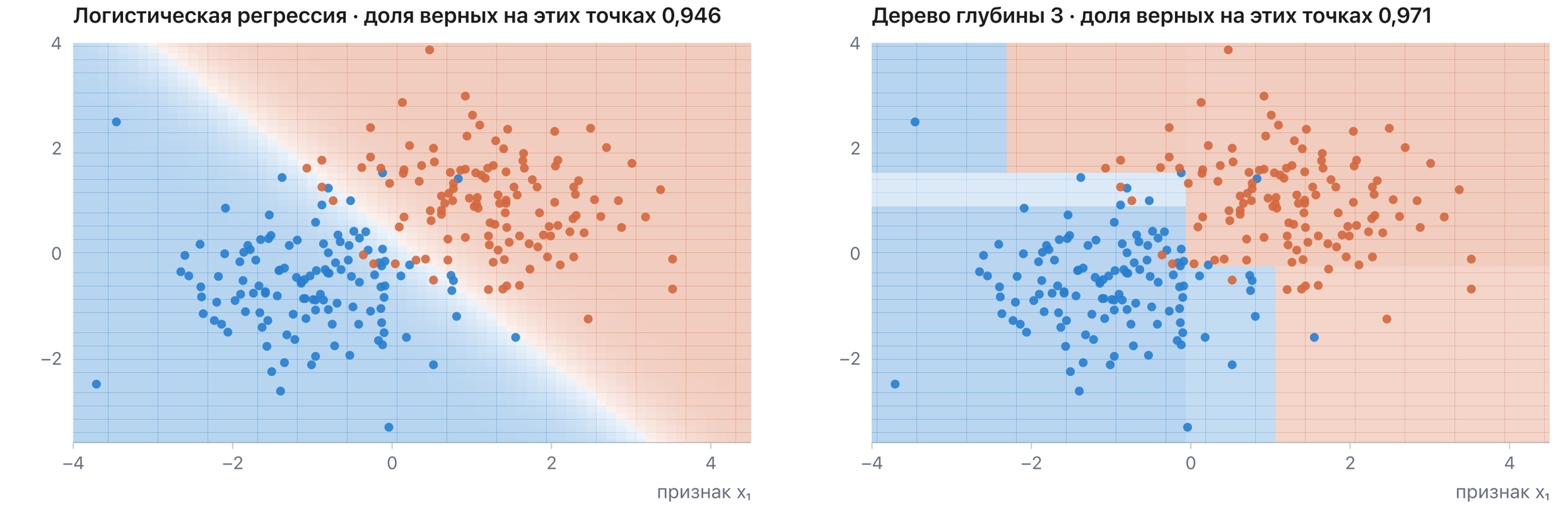
Дерево и прямая на одних данных



Внутри данных качество близкое. Дерево рисует ступеньки, прямая — общий тренд.

За границей train дерево повторяет ответ крайнего листа. Прямая продолжает тренд и уходит ниже нуля — экономичность не бывает отрицательной.

Граница классов: логрег и дерево



Учебная синтетика: два нормальных облака по 120 точек, seed = 1. Цвет фона — предсказанная вероятность класса 1; обе модели обучены на показанных точках.

Логрег проводит одну прямую и плавно меняет вероятность. Дерево режет плоскость на прямоугольники с постоянным ответом.

Дерево, линрег и логрег: сравнение

	Дерево, бустинг	Линейная регрессия	Логистическая регрессия
Задача	регрессия и классификация	регрессия	классификация
Что выдаёт	ответ листа или сумму листьев	число $w \cdot x + b$	вероятность $\sigma(w \cdot x + b)$
Форма	ступеньки, прямоугольники	прямая, плоскость — на исходных признаках	прямая граница — на исходных признаках
Что обучаем	признаки и пороги в узлах	веса: МНК или спуск	веса: спуск по log loss
Взаимодействия	находит само при глубине ≥ 2	нужен столбец $x_1 x_2$	
Масштаб признаков	не важен	важен для спуска и штрафа	
Вне диапазона train	константа	продолжает тренд	p уходит к 0 или 1
Защита от переобучения	глубина, шаг, число деревьев	L1- и L2-штраф, отбор признаков	
Как заглянуть внутрь	важности, SHAP, ICE	веса — при одинаковом масштабе и с оглядкой на похожие признаки	

С новыми признаками (x^2, $x_1 \cdot x_2$, one-hot) обе линейные модели рисуют кривые — они остаются линейными по весам.

Зачем линейные модели, если есть бустинг

Быстрый ориентир

Обучается за секунды, почти без настроек. Сразу видно, насколько задача вообще решаема.

Если сложная модель не обгоняет простую — повод проверить признаки, данные и схему проверки.

Разреженные признаки

Тексты, one-hot с тысячами категорий. Здесь линейные модели часто не хуже бустинга и намного быстрее.

Подробнее — на следующем слайде.

Тренд и мало данных

Когда зависимость близка к прямой, а объектов немного, две-три величины оценить проще, чем сотни листьев.

И прогноз продолжается за пределы train — если тренду можно верить.

Кроме того, линейный score внутри: у каждого нейрона нейросети, в стекинге поверх других моделей, в самой логистической регрессии.

Разреженные данные — сильная сторона линейных моделей

Тысячи столбцов, почти все значения — нули. Письмо как «мешок слов»: 20 000 столбцов «сколько раз встретилось слово», ненулевых в строке — около сотни.

ЛИНЕЙНАЯ МОДЕЛЬ

Хорошо

Score — сумма вкладов **только ненулевых** признаков: быстро даже на миллионах столбцов.

Каждое слово добавляет свой маленький вклад, и все вклады складываются.

`LogisticRegression` и `Ridge` принимают разреженную матрицу как есть.

БУСТИНГ НА ДЕРЕВЬЯХ

Часто хуже

Каждый узел смотрит на один столбец, а большинство слов встречается в единицах писем — на них трудно найти хороший порог.

Чтобы сложить свидетельства от сотни слов, нужны сотни узлов: обучается долго и легко переобучается.

```
vec = CountVectorizer()
X_train = vec.fit_transform(texts_train)   # разреженная матрица
model = LogisticRegression(C=1.0).fit(X_train, y_train)
```

ФИНАЛЬНЫЙ ОПРОС · 1 ИЗ 3

В test попалась машина тяжелее любой из train. Как поведут себя модели «масса → экономичность»?

A Обе выдадут среднее по train

✓ Дерево выдаст ответ крайнего листа, прямая продолжит снижать прогноз

C Дерево продолжит тренд, прямая выдаст константу

D Обе продолжат тренд: они обучены на одних данных

Дерево не знает ничего за последним порогом — там тот же лист. Линейная модель экстраполирует; хорошо это или плохо, зависит от того, продолжается ли тренд на самом деле.

ФИНАЛЬНЫЙ ОПРОС · 2 ИЗ 3

Доход — в рублях, возраст — в годах. Признаки не масштабировали. Какой модели это безразлично?

A kNN

B Ridge

C Логрегу, обучаемому градиентным спуском

✓ Градиентному бустингу на деревьях

Дерево сравнивает признак с порогом — от единиц результат не зависит. kNN считает расстояния, Ridge штрафует веса, спуску нужен общий шаг: всем трём масштаб важен.

ФИНАЛЬНЫЙ ОПРОС · 3 ИЗ 3

Скидка повышает продажи только в выходные. В таблице есть столбцы «скидка» и «выходной». Какая модель уловит это без новых признаков?

- ✓ Бустинг с деревьями глубины 2
- B Линейная регрессия
- C Ridge с правильно подобранным alpha
- D Линейная регрессия, если добавить столбцы скидка² и выходной²

Дерево глубины 2 задаёт два вопроса подряд: «выходной?» и «есть скидка?». Линейной модели нужен столбец «скидка × выходной» — квадраты отдельных признаков взаимодействие не описывают.

Вы открыли новую задачу. С чего начнёте?

Три постановки. Для каждой назовите первые три действия.

ЗАДАЧА A

Цена квартиры. 500 объектов, 12 числовых признаков. Метрика — RMSE.

Что предсказывать «в лоб»? Какую модель первой?

ЗАДАЧА B

Уйдёт ли клиент. 20 000 строк, 5% ушедших, есть категории и пропуски. Метрика — log loss.

Что сдавать: метки или вероятности?

ЗАДАЧА C

Спам или нет. Каждое письмо — 20 000 столбцов «встречается ли слово». Метрика — F1.

Что выбрать после обучения модели?

Порядок действий при виде задачи

01
Задача и метрика
Число или класс? Что считается ошибкой и что сдаём: метки или вероятности?

02
Разбиение
Train, validation, test — до любой подгонки. Для классов — стратификация.

03
Константа
Регрессия: среднее или медиана. Классы: доля класса 1 на train как вероятность, для accuracy — частый класс.

04
Признаки
Пропуски, one-hot, масштаб. Всё внутри Pipeline, обучение только на train.

05
Простая модель
Ridge или логрег. Быстро и видно, решаема ли задача.

06
Настройка
alpha или C, новые признаки, порог — по validation.

07
Сильная модель
Бустинг. Сравниваем по той же схеме и той же метрике.

08
Финал
Test — один раз. Записать, что пробовали и что получилось.

Спасибо за внимание!

Вопросы?

